Midterm 1

Graded

Student

Jackson Collins

Total Points

33 / 40 pts

Question 1

Indexing and slicing

4 / 7 pts

part c

✔ **– 1.5 pts** major/multiple issues with answer to part c

---

part e

✔ **– 1.5 pts** major/multiple issues with elements/characters selected by answer to part e

Question 2

List comprehensions

7.5 / 8 pts

part d

✔ **– 0.5 pts** missing brackets around individual elements of the list

Question 3

Tracing a recursive function

3 / 3 pts

✔ **– 0 pts** Correct

Question 4

Tracing function calls

4 / 4 pts

✔ **– 0 pts** Correct

Question 5

Writing a function 1

5 / 5 pts

✔ **– 0 pts** Correct

Question 6

Writing a function 2

3 / 5 pts

✔ **– 2 pts** incorrect attempt / does not attempt to handle cases in which there are no primes in the original list

Question 7

Conditional execution

3 / 3 pts

✔ **– 0 pts** Correct

Question 8

Writing a function 3

3.5 / 5 pts

recursive case

- ✔ **− 1.5 pts** one or more return statement does not correctly concatenate two lists

Midterm 1
CS 111
Spring 2026

Name Jackson Collins	Lecture A1 (12:20-1:10) _____ A2 (1:25-2:15) ✓
ID 053944815	

1. (7 points) Given the following assignment statements:

```
s = 'particular'
numbers = [7, 5, 3, 1, [4, 2]]
```

0 1 2 3 4

what is the value of each of the following? ***Make sure to clearly indicate the data type of each value.*** For example, strings should be surrounded by quotes.

a. `s[2]`

'r'

b. `s[ :5]`

'parti'

c. `numbers[-1: ]` `# note the variable!`

[[4,2],1,3,5,7]

d. `len(numbers) % 2 == 0`

False

len(numbers) = 5
5 % 2 = 1

e. `s[-3: :-2]` `# note the variable!`

[1,5]

2. (8 points) Evaluate each of the following:

a. `[y+1 for y in [1, 3, 5, 7]]`

[2,4,6,8]

b. `[x+'?' for x in 'why go']`

['w?','h?','y?',' ?','g?','o?']

c. `[y for y in range(4, 10) if y//2 >= 2.5]`

4,5,6,7,8,9

[6, 7, 8, 9]

d. `[[len(s)-1] for s in ['we', 'code', 'now']]`

[1, 3, 2]

3. (3 points) Consider the following recursive function:

```
def mystery(s):
    if s == '':
        return 0
    elif s[0] == 'r':
        return 1
    else:
        myst_rest = mystery(s[1:])
        return myst_rest + 3
```

a. Trace the execution of the function call `mystery('part')`. You may use any reasonable approach, provided that you show *all* of the recursive calls. You are ***not*** required to show the frames.

mystery('part')
9+3
mystery('art')
1+3
mystery('rt')
1

b. What is the value returned by `mystery('part')`?

7

4. (4 points) What is printed by the following program?

```
def hello(a, b):
    print('hello', a, b)
    b = a - 1
    a = goodbye(b)
    print('hello', a, b)
    return a

def goodbye(a):
    b = a * 2
    return b

a = 5
b = 2
b = hello(a, b)
print(a, b)
hello(b, a)
print(a, b)
```

Put the output below:

hello 5 2
hello 8 4
5 8
hello 8 5
hello 14 7
5 8

email

5. (5 points) Write a recursive function `transform(s)` that takes a string `s` of 0 or more **lower-case letters.** It should return a string containing all characters in `s` that are vowels (a, e, i, o or u), in the same order in which they appear in `s`. For example:

   - `transform('recursion')` should return the string `'euio'`
   - `transform('terrier')` should return the string `'eie'`
   - `transform('xyz')` should return the empty string (`''`) because there are no vowels in `'xyz'`

   Important: You *must* use recursion. You may *not* use a list comprehension, a loop, or any construct or function that we haven't discussed in lecture. However, you *are* allowed to use any operator or function that we *have* discussed!

```
def transform(s):
    if len(s) == 0:
        return ''
    if len(s) == 1 and s in 'aeiou':
        return s[0]

    else:
        rest = transform(s[1:])
        if s[0] in 'aeiou':
            return s[0] + rest

        else:
            return rest
```

6. (5 points) Assume that you have been given a helper function called `is_prime(x)` that takes an integer `x` and returns `True` if `x` is prime (i.e., an integer greater than 1 whose only factors are 1 and itself) and `False` otherwise. For example, `is_prime(3)` returns `True` because the only factors of 3 are 1 and 3, and `is_prime(8)` returns `False` because 8 has other factors besides 1 and 8.

   Write a function `find(vals)` that takes a list `vals` of 1 or more positive integers and returns the smallest prime number (if any) in `vals`. If the list has no prime numbers, the function should return -1. For example:

   - `find([4, 7, 5, 15])` should return 5 (note that 4 is *not* a prime number, so 5 is the smallest prime number in the list)
   - `find([8, 6, 10, 15])` should return -1 because there are no prime numbers in that list.

   Important: You *must* use the `is_prime()` function (which you do ***not*** need to write), along with a list comprehension. You are also welcome to use one or more of the built-in functions discussed in lecture, although doing so is not required. You may *not* use recursion, a loop, or any construct or function not discussed in lecture.

```
def find(vals):
    prime = [x for x in vals if is_prime(x) == True]
    small = min(prime)
    return small
```

7. What is printed by the following Python program? (3 points)

```
val = 14
if val > 25 or val <= 20:
    print('tar')
    if val % 5 == 4:
        print('tip')
if val != 10:
    print('two')
    if val * 2 < 20:
        print('ten')
elif val // 2 == 7:
    print('tar')
```

Put the output below:

tar
tip
two

email

8. Write a recursive function `process(vals1, vals2)` that takes two lists containing 0 or more integers.
It should return a new list containing, for each position in the original lists, the larger of the two elements at that position. If the two elements at a given position are the same, you can include either one of them. For example:

- `process([2,3,5,6], [4,1,5,8])` should return `[4,3,5,8]` because:
  - of the two elements in position 0 (2 and 4), 4 is the larger element
  - of the two elements in position 1 (3 and 1), 3 is the larger element
  - the two elements in position 2 are the same (5 and 5)
  - of the two elements in position 3 (6 and 8), 8 is the larger element
- `process([8,6], [4,9])` should return `[8,9]` because 8 > 4 and 9 > 6
- `process([], [])` should return `[]`

If one list is longer than the other, its extra values should be included at the end of the returned list. For example, `process([8,6], [4,9,3,5])` should return `[8,9,3,5]`.

Important: You *must* use recursion. You may *not* use a list comprehension, a loop, or any construct or function that we haven't discussed in lecture. In addition, **for this function only**, you may **not** use the built-in `min` or `max` functions.

```
def Process(vals1, vals2):
    if (len(vals1) == ∅ and (len(vals2) == ∅:
        return ∅
    elif len(vals2) == ∅ :
        return vals1
    elif (len(vals1) == ∅ :
        return vals2
    else:
        rest = Process(vals1[1:], vals2[1:])
        if vals1[∅] > vals2[∅] :
            return vals1[∅] + rest
        elif vals2[∅] > vals1[∅]:
            return vals2[∅] + rest
        elif vals1[∅] == vals2[∅]:
            return vals1[∅] + rest
        else:
            return rest
```

Blank page for extra work;
do not detach!